

Musaliar College of Engineering & Technology

MUSALIAR COLLEGE P.O., PATHANAMTHITTA - 689 653



LABORATORY RECORD

Certified that this is the Bonafide Record of the work done by
Sri/Smt.....of.....Semester Class
of (Roll No.....)
of.....Branch
in the.....Laboratory
during the academic year 2024 - 2026

Name of Examination:

Internal Examiner

External Examiner

Staff in-charge

DEPARTMENT OF COMPUTER APPLICATIONS

VISION

“To produce competent and dynamic professionals in the field of Computer Applications to thrive and cater the changing needs of the society through research and education”.

MISSION

To impart high quality technical education and knowledge in Computer Applications. To introduce moral, ethical and social values to Computer Application students. To establish industry institute interaction to enhance the skills of Computer Application students. To promote research aimed towards betterment of society.

INDEX

SL. NO	DATE	NAME OF EXPERIMENT	PAGE NO	COURSE OUTCOME	REMARKS
1	31-07-25	Review of python programming, Matrix operations, Data Visualization	5-20	CO1	
2	07-08-25	Data handling using pandas	21-22	CO1	
3	07-08-25	K-NN Classification using public domain dataset	23-24	CO2	
4	07-08-25	K-NN Classification using Custom Dataset	25-27	CO2	
5	14-08-25	Naive Bayes Classification using Public Domain Dataset	28-29	CO2	
6	14-08-25	Naive Bayes Classification using Custom Dataset	30-32	CO2	
7	21-08-25	Linear Regression using Public Domain Dataset	33-34	CO2	
8	21-08-25	Linear Regression using Custom Dataset	35-36	CO2	
9	11-09-25	Multiple Regression using Public Domain Dataset	37-39	CO2	
10	11-09-25	Multiple Regression using Custom Dataset	40-42	CO2	

11	18-09-25	Support Vector Machine	43-44	CO3	
12	18-09-25	Decision Trees using Public Domain Dataset	45-46	CO3	
13	18-09-25	Decision Trees using Custom Dataset	47-49	CO3	
14	25-09-25	K-Means Clustering using Public Domain Dataset	50-51	CO3	
15	25-09-25	K-Means Clustering using Custom Dataset	52-54	CO3	
16	09-10-25	Convolutional Neural Network using Public Domain Dataset	55-56	CO4	
17	09-10-25	Convolutional Neural Network using dataset	57-59	CO4	
18	16-10-25	Web Crawler and Scrapping web pages	60-61	CO5	
19	16-10-25	NLP Implementation using NLTK	62-64	CO5	

PROGRAM - 1

AIM:

Review of python programming, Matrix operations, Data Visualization using matplotlib.

Python Programming

AIM: Consider the following data:

Programming languages:	Java	Python	PHP	JavaScript	C#	C++
Popularity	22.2	17.6	8.8	8	77	6.7

Write a Python programming to draw a bar chart as well as a pie chart showing the popularity of programming Languages.

ALGORITHM

1. **Start**
2. Define the data:
 - Create a list languages containing programming languages.
 - Create a list popularity containing corresponding popularity percentages.
3. Set the figure size for the plot.
4. **Create the Bar Chart:**
 - Use `plt.subplot(1, 2, 1)` to set up the first subplot.
 - Plot a bar chart with languages on the x-axis and popularity on the y-axis.
 - Add titles, axis labels, and gridlines for clarity.
5. **Create the Pie Chart:**
 - Use `plt.subplot(1, 2, 2)` to set up the second subplot.
 - Plot a pie chart using popularity values and languages as labels.
 - Add the title for the pie chart.
6. Adjust the layout of the plots for proper spacing.
7. Display both charts using `plt.show()`.
8. **Stop**

PROGRAM

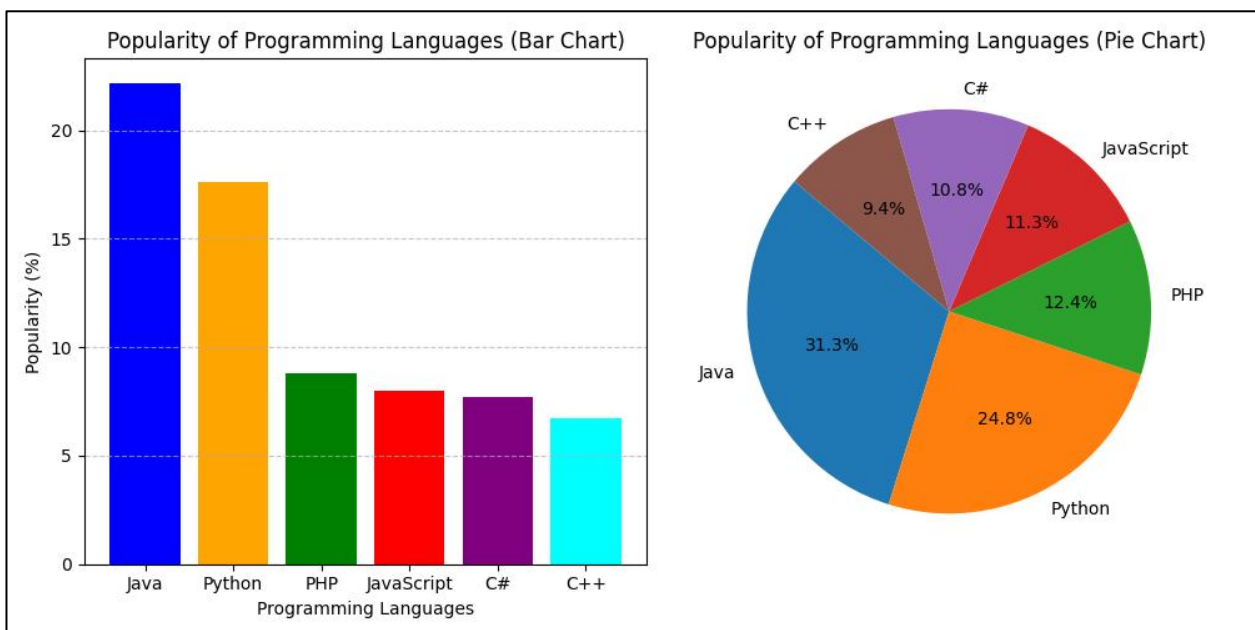
```
import matplotlib.pyplot as plt  
languages = ['Java', 'Python', 'PHP', 'JavaScript', 'C#', 'C++']
```

```

popularity = [22.2, 17.6, 8.8, 8.0, 7.7, 6.7]
plt.figure(figsize=(10, 5))
# ----- BAR CHART -----
plt.subplot(1, 2, 1)
plt.bar(languages, popularity, color=['blue', 'orange', 'green', 'red', 'purple', 'cyan'])
plt.title('Popularity of Programming Languages (Bar Chart)')
plt.xlabel('Programming Languages')
plt.ylabel('Popularity (%)')
plt.grid(axis='y', linestyle='--', alpha=0.7)
# ----- PIE CHART -----
plt.subplot(1, 2, 2)
plt.pie(popularity, labels=languages, autopct='%1.1f%%', startangle=140)
plt.title('Popularity of Programming Languages (Pie Chart)')
plt.tight_layout()
plt.show()

```

OUTPUT



RESULT: Program executed successfully and output is verified.

Matrix operations

AIM

Write programs using Matrix

ALGORITHM

1. Start
2. Define Matrices
 - Define matrix A and matrix B as 3×3 NumPy arrays.
3. Display Matrices
 - Print matrix A and matrix B.
4. Matrix Addition
 - Add matrices A and B element-wise using $A + B$.
 - Print the result of the addition.
5. Matrix Subtraction
 - Subtract matrix B from matrix A element-wise using $A - B$.
 - Print the result of the subtraction.
6. Matrix Multiplication
 - Multiply matrices A and B using `np.dot(A, B)`.
 - Print the result of the multiplication.
7. Element-wise Multiplication
 - Perform element-wise multiplication of matrices A and B using $A * B$.
 - Print the result of the element-wise multiplication.
8. Transpose of Matrix A
 - Find the transpose of matrix A using `A.T`.
 - Print the transpose of matrix A.
9. Determinant of Matrix A
 - Compute the determinant of matrix A using `np.linalg.det(A)`.
 - Print the determinant of matrix A.
10. Inverse of Matrix A
 - Check if the determinant of A is not zero.
 - If non-zero, compute and print the inverse of A using `np.linalg.inv(A)`.
 - If determinant is zero, print that the inverse does not exist.
11. Stop

PROGRAM

```
import numpy as np
A = np.array([[1, 2, 3],[4, 5, 6],[7, 8, 9]])
B = np.array([[9, 8, 7], [6, 5, 4],[3, 2, 1]])
print("Matrix A:\n", A)
print("\nMatrix B:\n", B)
add = A + B
print("\nAddition of A and B:\n", add)
sub = A - B
print("\nSubtraction of A and B:\n", sub)
mul = np.dot(A, B)
print("\nMultiplication of A and B:\n", mul)
element_mul = A * B
print("\nElement-wise Multiplication of A and B:\n", element_mul)
transpose_A = A.T
print("\nTranspose of Matrix A:\n", transpose_A)
det_A = np.linalg.det(A)
print("\nDeterminant of Matrix A: ", det_A)
try:
    inv_A = np.linalg.inv(A)
    print("\nInverse of Matrix A:\n", inv_A)
except np.linalg.LinAlgError:
    print("\nMatrix A is singular; inverse does not exist.")
```


OUTPUT

```
Matrix A:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

Matrix B:
[[9 8 7]
 [6 5 4]
 [3 2 1]]

Addition of A and B:
[[10 10 10]
 [10 10 10]
 [10 10 10]]

Subtraction of A and B:
[[-8 -6 -4]
 [-2  0  2]
 [ 4  6  8]]

Multiplication of A and B:
[[ 30  24  18]
 [ 84  69  54]
 [138 114  90]]
```

```
Element-wise Multiplication of A and B:
[[ 9 16 21]
 [24 25 24]
 [21 16  9]]

Transpose of Matrix A:
[[1 4 7]
 [2 5 8]
 [3 6 9]]

Determinant of Matrix A:  0.0

Matrix A is singular; inverse does not exist.
```

RESULT: Program executed successfully and output is verified.

AIM

Write a NumPy program to create an element-wise comparison (greater, greater_equal, less and less_equal) of two given arrays.

ALGORITHM

1. Start
2. Import the NumPy library.
3. Create two arrays a and b with numeric values.
4. Display both arrays.
5. Compare the elements of arrays a and b using:
 6. `np.greater(a, b)` → checks if elements of a are greater than b.
 7. `np.greater_equal(a, b)` → checks if elements of a are greater than or equal to b.
 8. `np.less(a, b)` → checks if elements of a are less than b.
 9. `np.less_equal(a, b)` → checks if elements of a are less than or equal to b.
10. Display the results of all comparisons.
11. Stop

PROGRAM

```
import numpy as np
a = np.array([5, 10, 15, 20])
b = np.array([10, 10, 5, 25])
print("Array A:", a)
print("Array B:", b)
print("\nGreater (A > B):")
print(np.greater(a, b))
print("\nGreater Equal (A >= B):")
print(np.greater_equal(a, b))
print("\nLess (A < B):")
print(np.less(a, b))
print("\nLess Equal (A <= B):")
print(np.less_equal(a, b))
```

OUTPUT

```
Array A: [ 5 10 15 20]
Array B: [10 10  5 25]

Greater (A > B):
[False False  True False]

Greater Equal (A >= B):
[False  True  True False]

Less (A < B):
[ True False False  True]

Less Equal (A <= B):
[ True  True False  True]
```

RESULT: Program executed successfully and output is verified.

AIM

Write a NumPy program to compute sum of all elements, sum of each column and sum of each row of a given array.

ALGORITHM

1. Start
2. Import the NumPy library.
3. Create a 3×3 NumPy array named arr.
4. Display the array.
5. Find the sum of all elements using np.sum(arr).
6. Display the total sum.
7. Find the sum of each column using np.sum(arr, axis=0).
8. Display the column-wise sum.
9. Find the sum of each row using np.sum(arr, axis=1).
10. Display the row-wise sum.
11. Stop

PROGRAM

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Array:")
print(arr)
total_sum = np.sum(arr)
print("\nSum of all elements:", total_sum)
col_sum = np.sum(arr, axis=0)
print("Sum of each column:", col_sum)
row_sum = np.sum(arr, axis=1)
print("Sum of each row:", row_sum)
```

OUTPUT

```
Array:  
[[1 2 3]  
 [4 5 6]  
 [7 8 9]]  
  
Sum of all elements: 45  
Sum of each column: [12 15 18]  
Sum of each row: [ 6 15 24]
```

RESULT: Program executed successfully and output is verified.

Data Visualization

AIM

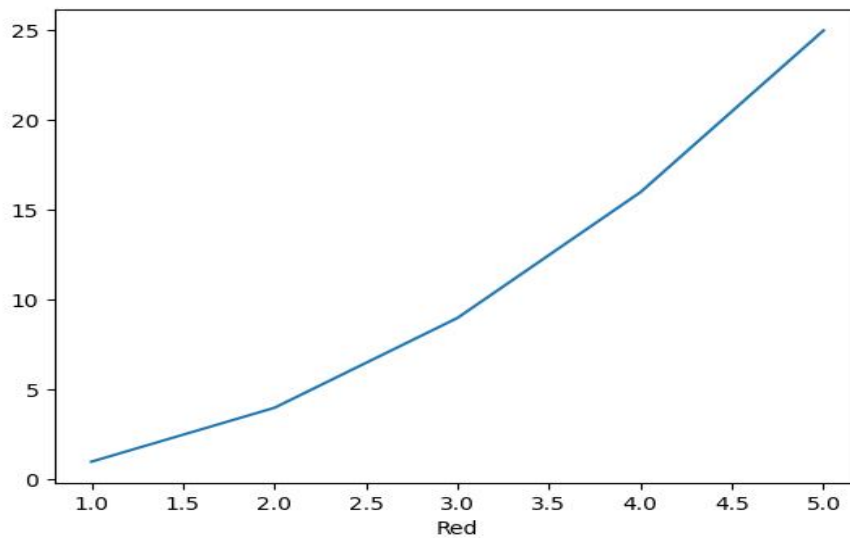
Write program for data visualization using matplotlib

Graph

PROGRAM

```
import matplotlib.pyplot as plt
x=[1,2,3,4,5]
y=[1,4,9,16,25]
plt.xlabel("Red")
plt.plot(x,y)
plt.show()
```

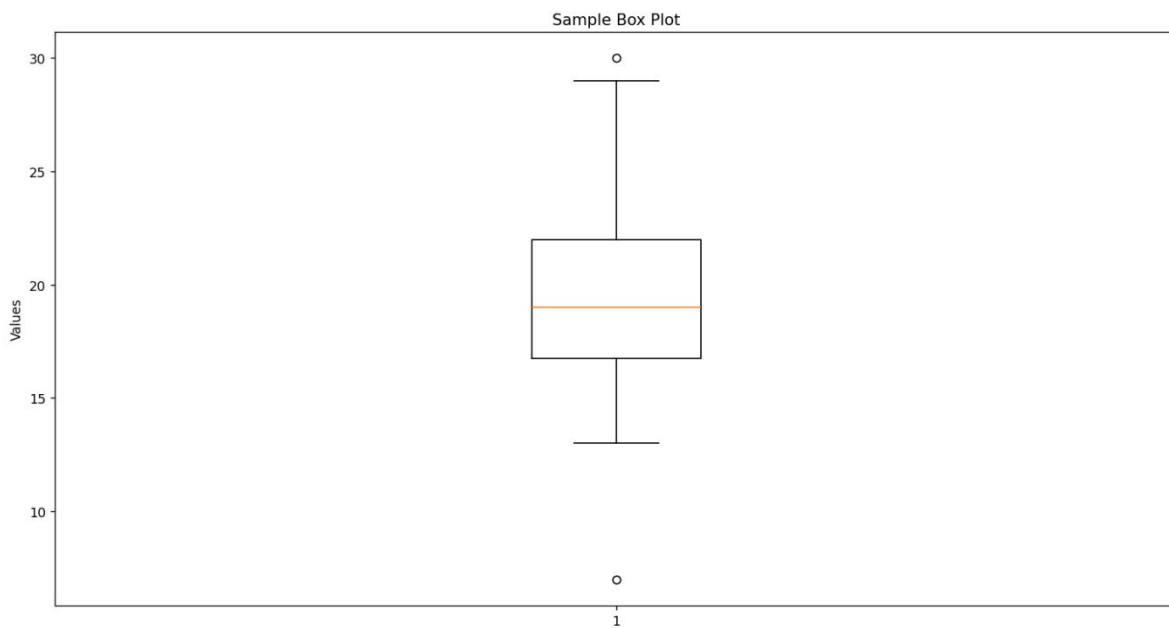
OUTPUT



Box Plot

```
import matplotlib.pyplot as plt
data = [7, 15, 13, 17, 19, 21, 22, 18, 15, 17, 20, 23, 25, 30, 29, 18, 16, 19, 21, 22]
plt.boxplot(data)
plt.title('Sample Box Plot')
plt.ylabel('Values')
plt.show()
```

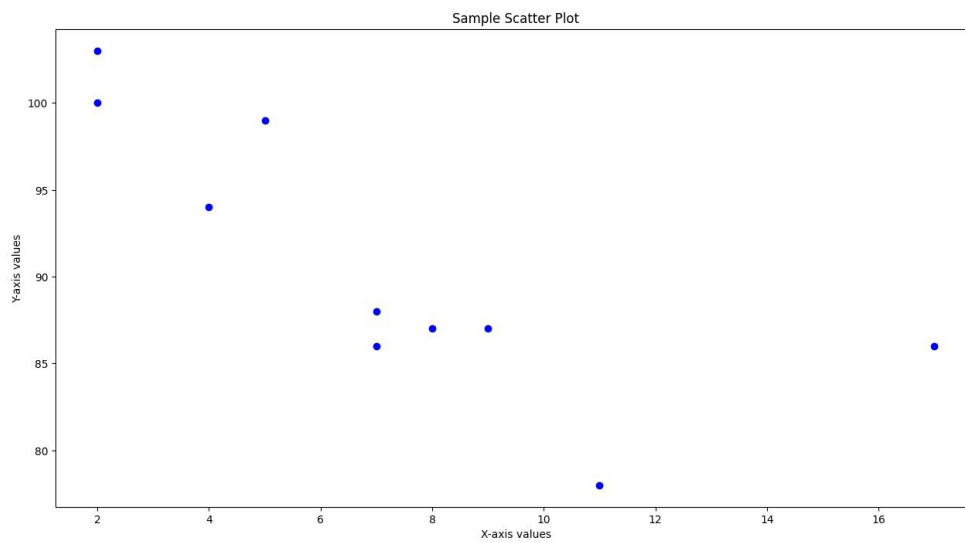
OUTPUT



Scatter Plot

```
import matplotlib.pyplot as plt
x = [5, 7, 8, 7, 2, 17, 2, 9, 4, 11]
y = [99, 86, 87, 88, 100, 86, 103, 87, 94, 78]
plt.scatter(x, y, color='blue', marker='o')
plt.title('Sample Scatter Plot')
plt.xlabel('X-axis values')
plt.ylabel('Y-axis values')
plt.show()
```

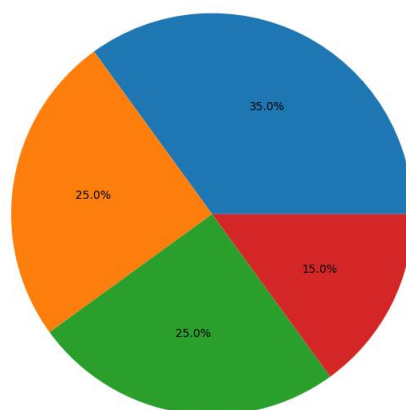
OUTPUT



Pie Chart

```
import matplotlib.pyplot as plt
import numpy as np
y=np.array([35,25,25,15])
plt.pie(y,autopct='%1.1f%%')
plt.show()
```

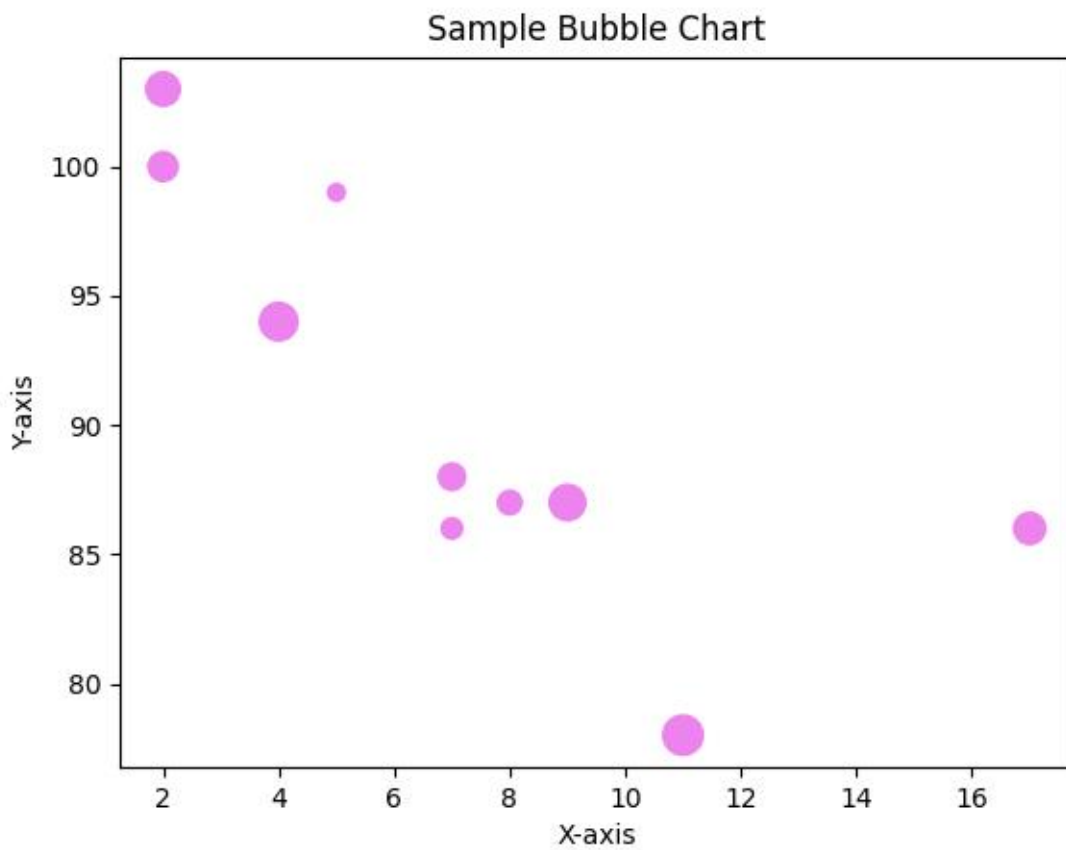
OUTPUT



Bubble Plot

```
import matplotlib.pyplot as plt
x = [5, 7, 8, 7, 2, 17, 2, 9, 4, 11]
y = [99, 86, 87, 88, 100, 86, 103, 87, 94, 78]
sizes = [40, 60, 80, 100, 120, 140, 160, 180, 200, 220]
plt.scatter(x, y, s=sizes, alpha=1, color='violet')
plt.title('Sample Bubble Chart')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```

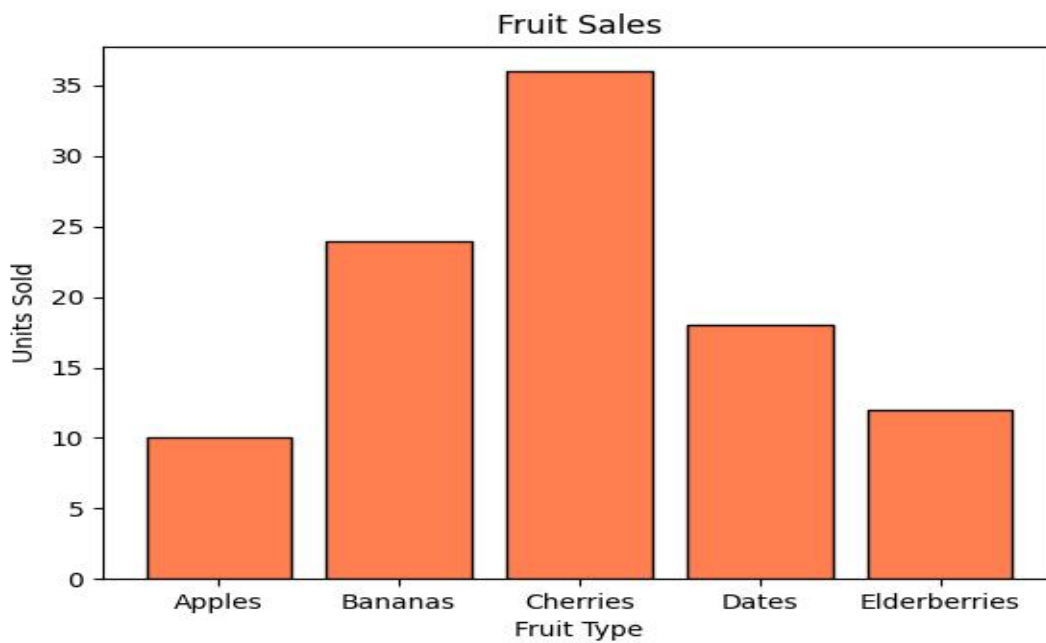
OUTPUT



Bar Chart

```
import matplotlib.pyplot as plt
categories = ['Apples', 'Bananas', 'Cherries', 'Dates', 'Elderberries']
values = [10, 24, 36, 18, 12]
plt.bar(categories, values, color='coral', edgecolor='black')
plt.title('Fruit Sales')
plt.xlabel('Fruit Type')
plt.ylabel('Units Sold')
plt.show()
```

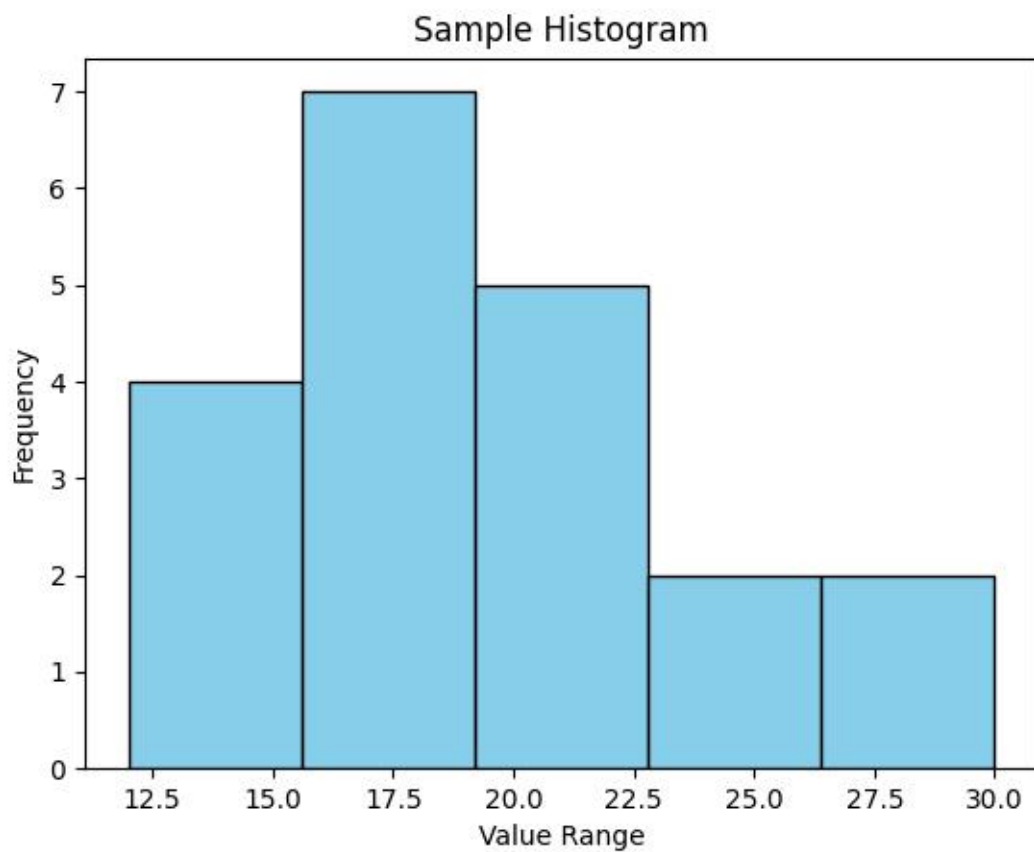
OUTPUT



Histogram

```
import matplotlib.pyplot as plt
data = [12, 15, 13, 17, 19, 21, 22, 18, 15, 17, 20, 23, 25, 30, 29, 18, 16, 19, 21, 22]
plt.hist(data, bins=5, color='skyblue', edgecolor='black')
plt.title('Sample Histogram')
plt.xlabel('Value Range')
plt.ylabel('Frequency')
plt.show()
```

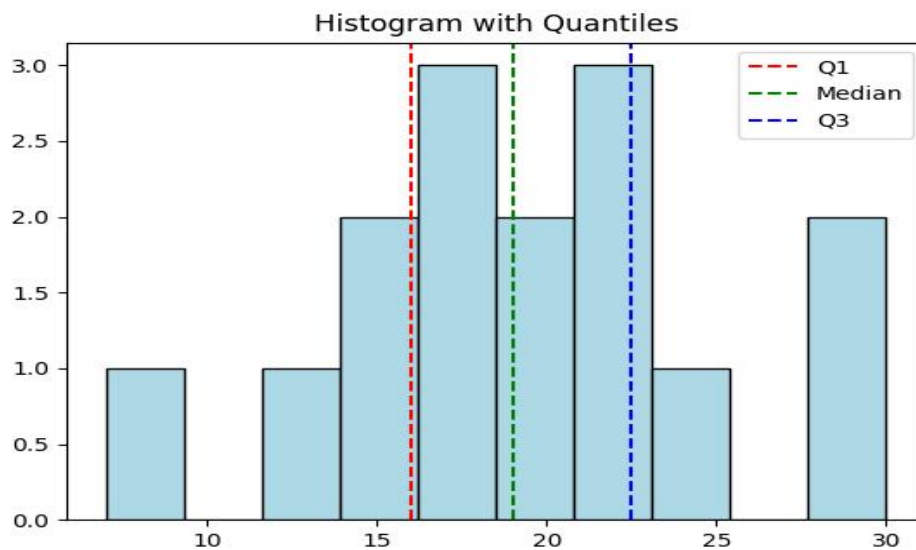
OUTPUT



Quantile

```
import numpy as np
import matplotlib.pyplot as plt
data = [7, 15, 13, 17, 19, 21, 22, 18, 15, 17, 20, 23, 25, 30, 29]
q1, median, q3 = np.quantile(data, [0.25, 0.5, 0.75])
plt.hist(data, color='lightblue', edgecolor='black')
plt.axvline(q1, color='red', linestyle='--', label='Q1')
plt.axvline(median, color='green', linestyle='--', label='Median')
plt.axvline(q3, color='blue', linestyle='--', label='Q3')
plt.legend()
plt.title('Histogram with Quantiles')
plt.show()
```

OUTPUT



RESULT: Program executed successfully and output is verified.

PROGRAM - 2

AIM:Data handling using pandas:

Create a student table with columns Roll.no, Name, age, marks using pandas and do the following

- a. select the top 2 rows
- b. filter data based on some condition with mark greater than 80
- c. filter in names first name start with N then remaining.

PROGRAM

```
import pandas as pd
data = {
    'Roll.No': [1, 2, 3, 4, 5],
    'Name': ['Nithin', 'Asha', 'Naveen', 'John', 'Neha'],
    'Age': [20, 21, 22, 20, 23],
    'Marks': [85, 76, 92, 67, 88]
}
students = pd.DataFrame(data)
print(" Student Table:")
print(students)
print("\n(a) Top 2 Rows:")
print(students.head(2))
print("\n(b) Students with Marks greater than 80:")
print(students[students['Marks'] > 80])
names_starting_with_N = students[students['Name'].str.startswith('N')]
remaining_names = students[~students['Name'].str.startswith('N')]
print("\n(c1) Students whose names start with 'N':")
print(names_starting_with_N)
print("\n(c2) Remaining Students:")
print(remaining_names)
```

OUTPUT

🔑 Student Table:

	Roll.No	Name	Age	Marks
0	1	Nithin	20	85
1	2	Asha	21	76
2	3	Naveen	22	92
3	4	John	20	67
4	5	Neha	23	88

(a) Top 2 Rows:

	Roll.No	Name	Age	Marks
0	1	Nithin	20	85
1	2	Asha	21	76

(b) Students with Marks greater than 80:

	Roll.No	Name	Age	Marks
0	1	Nithin	20	85
2	3	Naveen	22	92
4	5	Neha	23	88

(c1) Students whose names start with 'N':

	Roll.No	Name	Age	Marks
0	1	Nithin	20	85
2	3	Naveen	22	92
4	5	Neha	23	88

(c2) Remaining Students:

	Roll.No	Name	Age	Marks
1	2	Asha	21	76
3	4	John	20	67

RESULT: Program executed successfully and output is verified.

PROGRAM - 3

K-NN Classification Using Public Domain Dataset

AIM

Program to implement k-NN classification using any standard dataset available in the public domain and find the accuracy of the algorithm.

ALGORITHM

1. Import KNeighborsClassifier and load_iris from sklearn.
2. Load the Iris dataset.
3. Assign X = irisdata.data and Y = irisdata.target.
4. Split the dataset into training and testing sets.
5. Create a KNN model with one neighbor using KNeighborsClassifier(n_neighbors=1).
6. Train the model using fit(X_train, Y_train).
7. Predict the class labels for the test data using predict(X_test).
8. Predict the class for a new data sample and display the result.

PROGRAM

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
irisdata=load_iris()
print(irisdata.data)
x=irisdata.data
y=irisdata.target
x_train,x_test,y_train,y_test=train_test_split(x,y)
knn=KNeighborsClassifier(n_neighbors=1)
knn.fit(x_train,y_train)
print(x_test)
print(knn.predict(x_test))
print(y_test)
result=knn.predict([[2,4,6,2]])
print(irisdata.target_names[result])
```

OUTPUT

```
[[5.1 3.5 1.4 0.2]
 [4.9 3. 1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5. 3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [5.1 2.5 3. 1.1]
 [5.8 2.7 5.1 1.9]
 [6.2 3.4 5.4 2.3]
 [6.1 3. 4.9 1.8]
 [7.4 2.8 6.1 1.9]
 [5.5 2.5 4. 1.3]]
[0 2 0 1 0 1 2 2 2 0 0 2 1 0 2 2 2 2 1 1 0 2 2 2 2 2 0 1 0 2 2 1 1 1 0 0 0
 1]
[0 2 0 1 0 1 2 2 2 0 0 2 1 0 2 2 2 2 2 1 0 2 2 2 2 2 0 1 0 2 2 2 1 1 0 0 0
 1]
['virginica']
```

RESULT: Program executed successfully and output is verified.

PROGRAM - 4

K-NN Classification using Custom Dataset

AIM

To implement the k-Nearest Neighbors (k-NN) classification algorithm using a custom dataset

Feature1	Feature2	Class
10	9	Fruit
1	4	Protein
10	1	Fruit
7	10	Vegetable
3	10	Vegetable
1	1	Protein
8	5	Fruit
3	7	Vegetable
3	6	Protein
7	3	Fruit

ALGORITHM

1. Import the required libraries like pandas, sklearn, matplotlib, etc.
2. Load the dataset using `pandas.read_excel()`
3. Define a function `train_knn_model()` that performs data splitting, model training, and prediction.
4. Split the dataset into features (X) and labels (y).
5. Divide the dataset into training and testing sets using `train_test_split()`.
6. Create the KNN model using `KNeighborsClassifier(n_neighbors = 3)`.
7. Train the model using the training data with `.fit()`.
8. Predict the classes for the test data using `.predict()`.
9. Evaluate the model by calculating Accuracy, Precision, and F1-Score.
10. Provide a new sample input and display its predicted class.
11. Print or visualize the evaluation results using matplotlib or print statements.

PROGRAM

```
import pandas as pd

from sklearn.neighbors import KNeighborsClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import precision_score, accuracy_score, f1_score
```

```

data=pd.read_excel('food_knn_dataset.xlsx')

print(data)

x=data[['Feature1','Feature2']]

y=data['Class']

x_train,x_test,y_train,y_test=train_test_split(x,y)

knn=KNeighborsClassifier(n_neighbors=3)

knn.fit(x_train,y_train)

y_pred=knn.predict(x_test)

accuracy=accuracy_score(y_test,y_pred)

print("accuracy:",accuracy)

precision=precision_score(y_test,y_pred,average='macro',zero_division=1)

print("precision:",precision)

f1=f1_score(y_test,y_pred,average='macro',zero_division=1)

print("f1:",f1)

new_sample=pd.DataFrame([[6,4]],columns=['Feature1','Feature2'])

new_prediction=knn.predict(new_sample)

print("\nTomato is a:",new_prediction)

```

OUTPUT

```
Feature1 Feature2 Class
0      10      9   Fruit
1       1      4 Protein
2      10      1   Fruit
3       7     10 Vegetable
4       3     10 Vegetable
5       1      1 Protein
6       8      5   Fruit
7       3      7 Vegetable
8       3      6 Protein
9       7      3   Fruit
accuracy: 0.6666666666666666
precision: 0.6666666666666666
f1: 0.5555555555555555
```

RESULT: Program executed successfully and output is verified.

PROGRAM - 5

Naive Bayes Classification using Public Domain Dataset

AIM

Program to implement Naive Bayes classification using any dataset Input:

ALGORITHM

1. Start
2. Import the required libraries such as `sklearn.datasets`, `sklearn.model_selection`, and `sklearn.naive_bayes`.
3. Load the Iris dataset using `load_iris()` and store features in `X` and labels in `y`.
4. Split the dataset into training and testing sets using `train_test_split()`.
5. Create a Gaussian Naive Bayes (GNB) model using `GaussianNB()`.
6. Train the model using the training data with `.fit()`.
7. Predict the class labels for the test data using `.predict()`.
8. Display the predicted results.
9. Give a new input sample and predict its class label using the trained model.
10. Find the accuracy using the `.score()` function.
11. Display the accuracy of the model.
12. Stop

PROGRAM

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
X,y=load_iris(return_X_y=True)
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.5,random_state=0)
gnb=GaussianNB()
y_pred=gnb.fit(X_train,y_train).predict(X_test)
print(y_pred)
x_new=[[5,5,4,4]]
y_new=gnb.fit(X_train,y_train).predict(x_new)
print("predicted output for [[5,5,4,4]]:",y_new)
```

```
print("Naive Bayes score:",gnb.score(X_test,y_test))
```

OUTPUT

```
[2 1 0 2 0 2 0 1 1 1 1 1 1 1 1 0 1 1 0 0 2 1 0 0 2 0 0 1 1 0 2 1 0 2 2 1 0
 1 1 1 2 0 2 0 0 1 2 2 1 2 1 2 1 1 2 1 2 1 0 2 1 1 1 1 2 0 0 2 1 0 0
 1]
predicted output for [[5,5,4,4]]: [2]
Naive Bayes score: 0.9466666666666667
```

RESULT: Program executed successfully and output is verified.

PROGRAM - 6

Naive Bayes Classification using Custom Dataset

AIM

To implement the Naive Bayes classification algorithm using a custom food dataset as shown below and predict the class of a new sample.

Feature1	Feature2	Class
10	9	Fruit
1	4	Protein
10	1	Fruit
7	10	Vegetable
3	10	Vegetable
1	1	Protein
8	5	Fruit
3	7	Vegetable
3	6	Protein
7	3	Fruit
6	4	Fruit

ALGORITHM

1. Import required libraries like pandas, train_test_split, GaussianNB, and accuracy_score.
2. Load the dataset using pandas.read_excel().
3. Split the dataset into features (X) and labels (y).
4. Divide the dataset into training and testing sets.
5. Create a Naive Bayes model using GaussianNB().
6. Train the model using fit() with the training data.
7. Predict the class labels for the test data.
8. Calculate the accuracy, precision, and F1-score using appropriate metrics.
9. Provide a new sample input and display its predicted class.

PROGRAM

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB
```

```

from sklearn.metrics import accuracy_score, precision_score, f1_score

data = pd.read_excel('food_naivebayes_dataset.xlsx')

print("Dataset Loaded Successfully:\n")

print(data)

X = data[['Feature1', 'Feature2']]

y = data['Class']

x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

gnb = GaussianNB()

gnb.fit(x_train, y_train)

y_pred = gnb.predict(x_test)

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(y_test, y_pred, average='macro', zero_division=1)

f1 = f1_score(y_test, y_pred, average='macro', zero_division=1)

print("\nModel Evaluation Metrics:")

print("Accuracy:", round(accuracy, 2))

print("Precision:", round(precision, 2))

print("F1-Score:", round(f1, 2))

x_new = pd.DataFrame([[6, 4]], columns=['Feature1', 'Feature2'])

y_new = gnb.predict(x_new)

print("\nPredicted class for [6, 4] (Tomato):", y_new[0])

```

OUTPUT

```
Dataset Loaded Successfully:

  Feature1  Feature2  Class
0         10         9   Fruit
1          1         4 Protein
2         10         1   Fruit
3          7        10 Vegetable
4          3        10 Vegetable
5          1         1 Protein
6          8         5   Fruit
7          3         7 Vegetable
8          3         6 Protein
9          7         3   Fruit
10         6         4   Fruit

Model Evaluation Metrics:
Accuracy: 0.75
Precision: 0.67
F1-Score: 0.6

Predicted class for [6, 4] (Tomato): Fruit
```

RESULT: Program executed successfully and output is verified.

PROGRAM - 7

Linear Regression using Public Domain Dataset

AIM

To implement Linear Regression on a public domain dataset and predict the output for a new input value.

ALGORITHM

1. Import the required libraries numpy and LinearRegression from sklearn.linear_model.
2. Create the input data array X and output data array y.
3. Create a Linear Regression model using LinearRegression().
4. Train the model using the fit(X, y) function.
5. Display the coefficient of the model using coef_.
6. Display the intercept of the model using intercept_.
7. Predict the output for a new input value using the predict() function.
8. Display the predicted result

PROGRAM

```
from sklearn.linear_model import LinearRegression
import numpy as np
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 5, 4, 5])
#Train model
model = LinearRegression().fit(X, y)
#Predict and show results
print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
print("Prediction for x=6:", model.predict([[6]])[0])
```

OUTPUT

```
PS C:\Users\MCETCSE\Desktop\dsm12> python linear2.py  
Coefficient: 0.6  
Intercept: 2.2  
Prediction for x=6: 5.8  
PS C:\Users\MCETCSE\Desktop\dsm12>
```

RESULT: Program executed successfully and output is verified.

PROGRAM - 8

Linear Regression using Custom Dataset

AIM

To implement Linear Regression using a dataset from an Excel file (aptitude.xlsx) to predict output values for new input scores.

x1	y1
95	85
85	95
80	70
70	65
60	70

ALGORITHM

1. Import necessary libraries such as pandas, matplotlib.pyplot, and LinearRegression from sklearn.linear_model.
2. Read the dataset from the Excel file (aptitude.xlsx) using read_excel().
3. Assign the independent variable X and the dependent variable y from the dataset.
4. Split the data into training and testing sets using train_test_split().
5. Create a Linear Regression model using LinearRegression().
6. Train the model using the training data with the fit() function.
7. Predict the output for the test data using the predict() function.
8. Accept a new input value, calculate the predicted output using the regression equation and display the result

PROGRAM

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import datasets, linear_model
from sklearn import metrics
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score
df=pd.read_excel('aptitude.xlsx')
```

```
X=df[['x1']]
y=df[['y1']]
x_train,x_test,y_train,y_test=train_test_split(X,y,test_size=0.3)
clf=LinearRegression()
clf.fit(x_train,y_train)
y_pred=clf.predict(x_test)
print(y_pred)
print("y=",clf.intercept_,"+",clf.coef_,"* x")
newvalue=float(input("enter the score:"))
y_pred=clf.intercept_+clf.coef_*newvalue
print(y_pred)
```

OUTPUT



```
PS C:\Users\NICEITCSE\Desktop\dsm12> python linear.py
[[83.33333333]
 [77.94871795]]
y= [40.25641026] + [[0.53846154]] * x
enter the score:80
[[83.33333333]]
```

RESULT: Program executed successfully and output is verified.

PROGRAM - 9

Multiple Regression using Public Domain Dataset

AIM

To implement Multiple Linear Regression using selected features from the diabetes dataset to predict disease progression and evaluate the model performance.

ALGORITHM

1. Start.
2. Import required libraries:
 - scikit-learn for regression
 - pandas for data handling
 - matplotlib for visualization
3. Load the diabetes dataset and convert it into a pandas DataFrame.
4. Select independent variables (features) and the dependent variable (target).
5. Split the dataset into training and testing sets using `train_test_split()`.
6. Create a Linear Regression model and train it with the training data using `.fit()`.
7. Print the regression equation (intercept + coefficients).
8. Predict the target values for the test data.
9. Evaluate the model using Mean Squared Error (MSE) and R^2 score.
10. Predict the target value for new input data.
11. Visualize actual vs predicted values using a scatter plot.
12. Stop.

PROGRAM

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.datasets import load_diabetes
from sklearn.metrics import mean_squared_error, r2_score
import pandas as pd
import matplotlib.pyplot as plt
diabetes = load_diabetes(as_frame=True)
```

```

df = diabetes.frame

print("Dataset Loaded Successfully!\n")

print(df.head())

X = df[['bmi', 'bp', 's1', 's5']]

y = df['target']

x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = LinearRegression()

model.fit(x_train, y_train)

print("\n Regression Equation:")

print(f'y = {model.intercept_:.2f}', end='')

for feature, coef in zip(X.columns, model.coef_):

    print(f' + ({coef:.2f} × {feature})', end='')

print("\n")

y_pred = model.predict(x_test)

mse = mean_squared_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)

print(" Model Performance Metrics:")

print("Mean Squared Error (MSE):", round(mse, 2))

print("R2 Score:", round(r2, 2))

new_data = pd.DataFrame([[0.05, 0.03, 0.02, 0.04]], columns=['bmi', 'bp', 's1', 's5'])

predicted_value = model.predict(new_data)

print("\n Predicted Target Value for New Data:", round(predicted_value[0], 2))

plt.figure(figsize=(6,5))

plt.scatter(y_test, y_pred, color='blue', s=60)

plt.xlabel("Actual Values")

plt.ylabel("Predicted Values")

plt.title("Actual vs Predicted (Multiple Linear Regression)")

```

```
plt.grid(True)
```

```
plt.show()
```

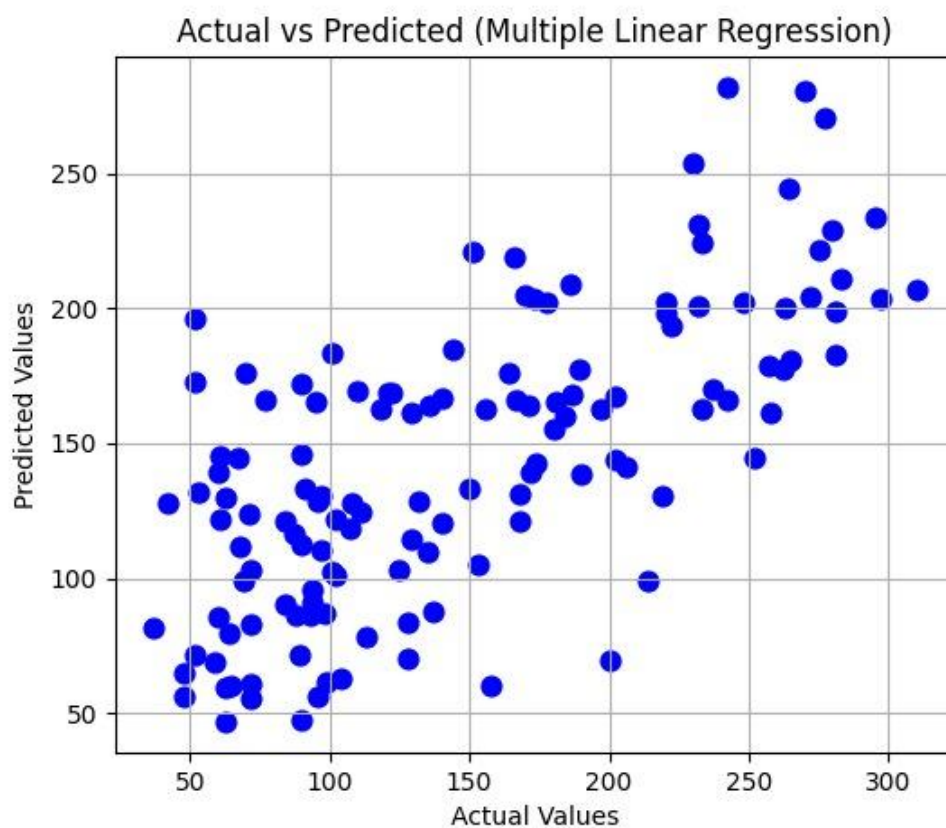
OUTPUT

```
2  0.085299  0.050680  0.044451 -0.005670 -0.045599 -0.034194 -0.032356 -0.002592  0.002861 -0.025930  141.0
3 -0.089063 -0.044642 -0.011595 -0.036656  0.012191  0.024991 -0.036038  0.034309  0.022688 -0.009362  206.0
4  0.005383 -0.044642 -0.036385  0.021872  0.003935  0.015596  0.008142 -0.002592 -0.031988 -0.046641  135.0
```

```
Regression Equation:
y = 150.84 + (669.56 x bmi) + (326.92 x bp) + (-235.49 x s1) + (561.27 x s5)
```

```
Model Performance Metrics:
Mean Squared Error (MSE): 2799.73
R2 Score: 0.48
```

```
Predicted Target Value for New Data: 211.87
```



RESULT: Program executed successfully and output is verified.

PROGRAM - 10

Multiple Regression using Custom Dataset

AIM

To implement Multiple Linear Regression using car specifications (Weight and Volume) to predict CO2 emissions.

No	Car	Model	Volume	Weight	CO2
1	TOYOTA	AYGO	1000	790	99
2	MITSUBISHI	SPACE STAR	1200	1160	95
3	SKODA	CITIGO	900	929	95
4	FIAT	FIAT 500	1500	865	90
5	MINI	COOPER	1000	1140	105
6	VW	UP!	1400	929	105
7	SKODA	FABIA	1500	1109	90
8	MERCEDES	A-CLASS	1500	1365	92
9	FORD	FIESTA	1500	1112	98
10	AUDI	A1	1600	1150	99

ALGORITHM

1. Start
2. Import the required libraries such as pandas, scikit-learn, and metrics.
3. Load the dataset (data.xlsx) into a pandas DataFrame.
4. Select independent variables (Weight, Volume) and the dependent variable (CO2).
5. Convert the features and target into NumPy arrays using .values.
6. Split the dataset into training and testing sets using train_test_split().
7. Display the training data and labels for verification.
8. Create a Linear Regression model using LinearRegression().
9. Train the model using .fit() on the training data.
10. Predict the CO2 emission for a new input value [2300, 1300].
11. Display the predicted CO2 value.
12. Find the model accuracy using .score() (R² value).
13. Display the R² score.
14. Stop

PROGRAM

```
from sklearn.model_selection import train_test_split

from sklearn import linear_model

import pandas as pd

from sklearn.metrics import r2_score

df = pd.read_excel('data.xlsx')

X = df[['Weight', 'Volume']]

y = df[['CO2']]

X = X.values

y = y.values

x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

print("Training Data (X):\n", x_train)

print("Training Labels (y):\n", y_train)

regr = linear_model.LinearRegression()

regr.fit(x_train, y_train)

predictedCO2 = regr.predict([[2300, 1300]])

print("\nPredicted CO2 for [2300,1300]:", predictedCO2)

score = regr.score(x_test, y_test)

print("\nModel R2 Score:", round(score, 2))
```

OUTPUT

```
Training Data (X):
[[1150 1600]
 [1109 1500]
 [1365 1500]
 [ 865 1500]
 [ 790 1000]
 [ 929 1400]]
Training Labels (y):
[[ 99]
 [ 95]
 [ 90]
 [ 95]
 [ 90]
 [ 90]
 [ 92]
 [ 90]
 [ 99]
 [105]]

Predicted CO2 for [2300,1300]: [[87.65719553]]

Model R2 Score: -0.72
```

RESULT: Program executed successfully and output is verified.

PROGRAM -11

Support Vector Machine

AIM

To implement Support Vector Machine (SVM) classification using the Breast Cancer dataset.

ALGORITHM

1. Import required libraries from sklearn: datasets, train_test_split, metrics, and svm.
2. Load the breast cancer dataset using datasets.load_breast_cancer().
3. Split the dataset into training and testing sets using train_test_split() with 30% test size.
4. Create a Support Vector Machine classifier with a linear kernel using svm.SVC(kernel='linear').
5. Train the classifier using the training data with fit().
6. Predict the class labels for the test data using predict().
7. Print the actual and predicted class labels.
8. Calculate and print performance metrics-accuracy, precision and recall

PROGRAM

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn import svm
cancer = datasets.load_breast_cancer()
x_train, x_test, y_train, y_test = train_test_split( cancer.data,
cancer.target, test_size=0.3, random_state=109)
clf = svm.SVC(kernel='linear')
clf.fit(x_train, y_train)
y_pred = clf.predict(x_test)
print("Actual values:", y_test)
print("Predicted values:", y_pred)
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
print("Precision:", metrics.precision_score(y_test, y_pred))
print("Recall:", metrics.recall_score(y_test, y_pred))
```

OUTPUT

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell
PS C:\DSML ANJU> python svm.py
Actual values: [1 1 0 0 1 0 1 1 1 0 0 0 1 0 1 1 0 0 1 0 1 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1 1
0 1 1 0 1 0 1 1 1 0 1 0 0 1 1 0 1 1 0 1 1 1 0 0 1 0 1 0 0 1 1 1 1 0 1 1 1
0 1 0 1 1 1 1 1 1 1 0 1 1 1 1 1 0 0 1 0 1 1 1 1 1 0 0 1 1 0 1 1 0 1 0 1 1
0 1 1 0 0 0 1 0 1 1 1 1 1 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 1 0 1 1 0 0 1 0
0 0 1 1 1 1 1 0 0 1 0 1 1 1 1 1 1 1 1 0 1 1 1]
Predicted values: [1 1 0 0 1 0 1 1 1 0 0 0 1 0 0 1 0 0 1 0 1 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1
0 1 1 0 1 0 1 1 1 1 1 0 0 1 1 0 1 1 0 1 1 1 0 0 1 0 1 0 0 1 1 1 1 0 1 1 1
0 1 0 1 1 1 1 1 1 1 0 0 1 1 1 1 0 0 0 0 1 1 1 1 1 0 0 1 0 0 1 1 0 1 0 1 1
0 1 1 0 0 0 1 0 1 1 1 1 1 0 1 0 1 0 1 0 0 0 1 0 1 1 0 1 1 1 0 1 1 0 0 1 0
0 0 1 1 1 1 1 0 0 1 0 1 1 1 1 1 1 1 1 0 1 1 1]
Accuracy: 0.9649122807017544
Precision: 0.9811320754716981
Recall: 0.9629629629629629
PS C:\DSML ANJU> 
```

RESULT: Program executed successfully and output is verified.

PROGRAM-12

Decision Trees using Public Domain Dataset

AIM

To implement a Decision Tree Classifier using the Iris dataset (public domain) and visualize the trained decision tree.

ALGORITHM

1. Import necessary libraries: load_iris from sklearn.datasets, DecisionTreeClassifier from sklearn.tree, and matplotlib.pyplot.
2. Load the Iris dataset using load_iris().
3. Assign the feature data to X and target labels to y.
4. Create a Decision Tree model using DecisionTreeClassifier().
5. Train the model using fit(X, y).
6. Plot the trained decision tree using tree.plot_tree() with filled colors, feature names, and class names.
7. Display the plotted decision tree using plt.show().

PROGRAM

```
from sklearn.datasets import load_iris

from sklearn import tree

import matplotlib.pyplot as plt

iris = load_iris()

X, y = iris.data, iris.target

clf = tree.DecisionTreeClassifier()

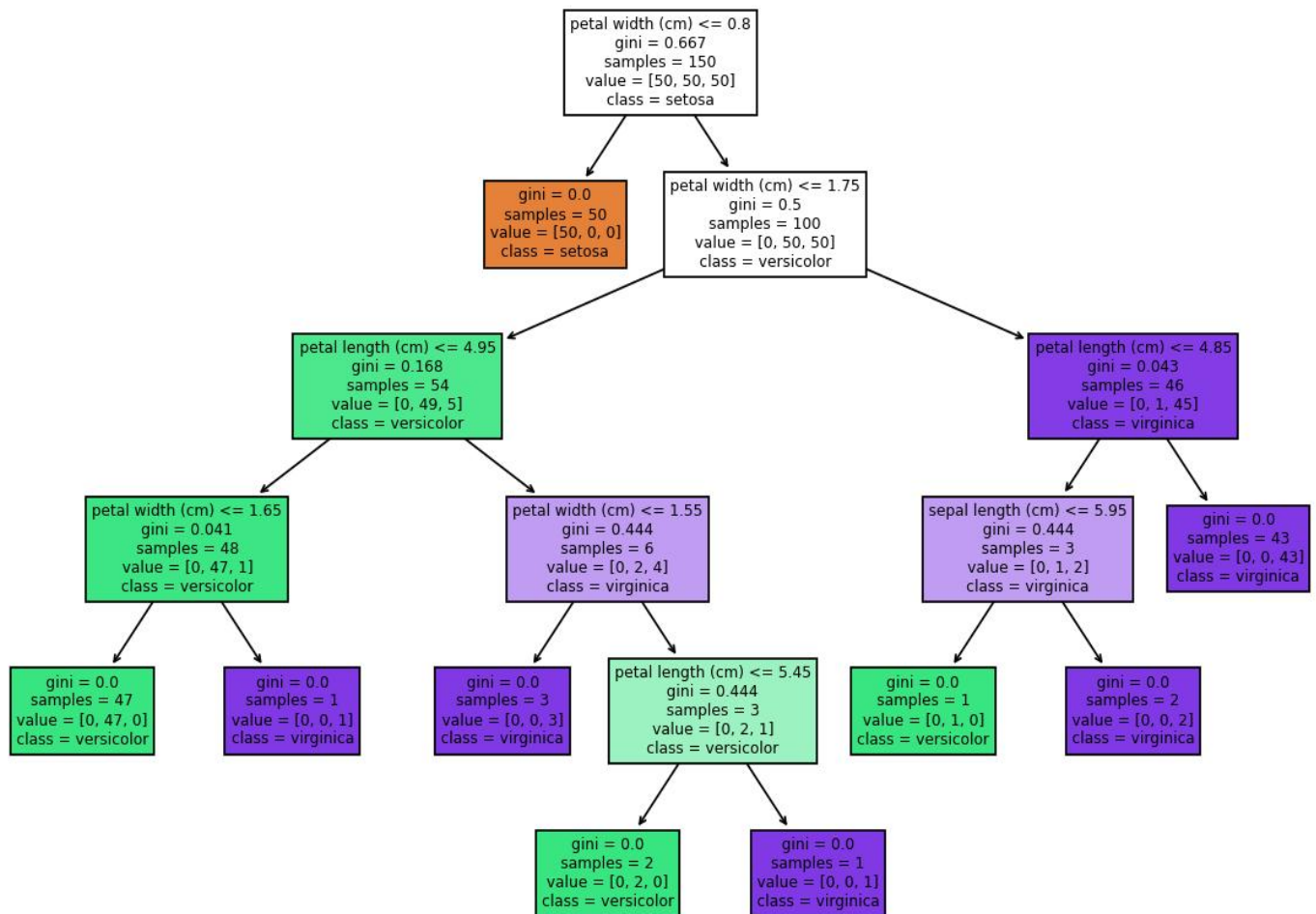
clf = clf.fit(X, y)

plt.figure(figsize=(12,8))

tree.plot_tree(clf, filled=True, feature_names=iris.feature_names, class_names=iris.target_names)

plt.show()
```

OUTPUT



RESULT: Program executed successfully and output is verified.

PROGRAM -13

Decision Trees using Custom Dataset

AIM

Implementation of Decision Trees Text classification using Support,

Data1.csv

name	gives_birth	aquatic_animal	aerial_animals	has_legs	class_label
human	1	0	0	1	mammal
python	0	0	0	0	reptile
salmon	0	1	0	0	fish
frog	0	2	0	1	amphibian
bat	1	0	1	1	bird
pigeon	0	0	1	1	bird
cat	1	0	0	1	mammal
shark	1	1	0	0	fish
turtle	0	2	0	1	amphibian
salamande	0	2	0	1	amphibian

ALGORITHM

1. Import the required libraries like pandas, sklearn, and matplotlib.
2. Load the dataset using `pandas.read_csv()`.
3. Separate the features (X) and the target (y).
4. Split the dataset into training and testing sets using `train_test_split()`.
5. Create a Decision Tree model using `DecisionTreeClassifier()`.
6. Train the model using `clf.fit(x_train, y_train)`.
7. Predict the test data using `clf.predict(x_test)`.
8. Calculate the accuracy score using `metrics.accuracy_score()`.
9. Print the predicted results and the accuracy score.

PROGRAM

```
from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn import tree, metrics

import pandas as pd

import matplotlib.pyplot as plt

df = pd.read_csv('data1.csv')

X = df[['gives_birth', 'aquatic_animal', 'aerial_animal', 'has_legs']]

y = df[['class_label']]

target_names = ['mammal', 'reptile', 'fish', 'amphibian', 'bird']

feature_names = ['gives_birth', 'aquatic_animal', 'aerial_animal', 'has_legs']

x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

print(x_train)

print(y_train)

clf = DecisionTreeClassifier()

clf.fit(x_train, y_train)

y_pred = clf.predict(x_test)

print(y_pred)

print(clf.predict(x_train))

print(y_train.values.ravel())

accuracy = metrics.accuracy_score(y_test, y_pred)

print("\nAccuracy Score:", round(accuracy * 100, 2), "%")
```


OUTPUT

```
gives_birth aquatic_animal aerial_animal has_legs
9           0             2           0       1
6           1             0           0       1
5           0             0           1       1
8           0             2           0       1
2           0             1           0       0
0           1             0           0       1
4           1             0           1       1
class_label
9  amphibian
6   mammal
5    bird
8  amphibian
2    fish
0   mammal
4    bird
['fish' 'mammal' 'amphibian']
['amphibian' 'mammal' 'bird' 'amphibian' 'fish' 'mammal' 'bird']
['amphibian' 'mammal' 'bird' 'amphibian' 'fish' 'mammal' 'bird']

Accuracy Score: 66.67 %
```

RESULT: Program executed successfully and output is verified.

PROGRAM -14

K-Means Clustering using Public Domain Dataset

AIM

To implement K-Means Clustering on the Iris dataset (public domain) to group the data points into clusters and visualize the cluster distribution along with centroids.

ALGORITHM

1. Import necessary libraries like numpy, pandas, matplotlib, KMeans.
2. Load the Iris dataset.
3. Select the features (Petal Length and Petal Width) as X.
4. Create a K-Means model with 3 clusters.
5. Fit the model and predict cluster labels.
6. Plot the data points colored by cluster.
7. Plot the cluster centroids.
8. Add titles, labels, legend, and display the plot.

PROGRAM

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

from sklearn.datasets import load_iris

iris = load_iris()

X = iris.data[:, [2, 3]]

df = pd.DataFrame(X, columns=['Petal Length', 'Petal Width'])

kmeans = KMeans(n_clusters=3, init='k-means++', random_state=42)

y_kmeans = kmeans.fit_predict(X)

plt.figure(figsize=(8, 6))
```

```

plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s=100, c='red', label='Cluster 1')
plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1], s=100, c='blue', label='Cluster 2')
plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1], s=100, c='green', label='Cluster 3')
plt.scatter(kmeans.cluster_centers_[0, 0], kmeans.cluster_centers_[0, 1], s=300, c='yellow',
marker='*', label='Centroids')

plt.title('K-Means Clustering on Iris Dataset')

plt.xlabel('Petal Length (cm)')
plt.ylabel('Petal Width (cm)')

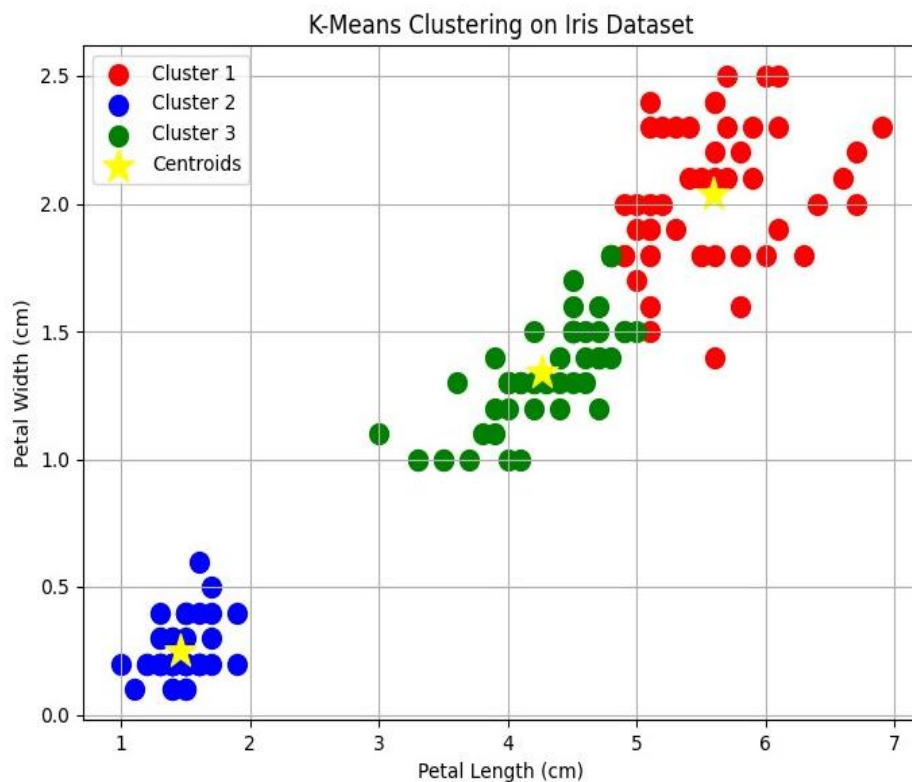
plt.legend()

plt.grid(True)

plt.show()

```

OUTPUT



RESULT: Program executed successfully and output is verified.

PROGRAM -15

K-Means Clustering using Custom Dataset

AIM

To implement the K-Means clustering algorithm using the Mall_Customers dataset and visualize customer clusters based on Annual Income and Spending Score.

Customer ID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
1	Male	19	15	39
2	Male	21	15	81
3	Female	20	16	6
4	Female	23	16	77
5	Female	31	17	40
6	Female	22	17	76
7	Female	35	18	6
8	Female	23	18	94
9	Male	64	19	3
10	Male	30	19	72
11	Male	67	20	14
12	Male	35	20	99
13	Female	58	21	15
14	Female	24	21	81
15	Male	37	23	13
16	Female	22	23	79
17	Female	35	24	35
18	Male	20	24	73
19	Female	52	25	5
20	Male	35	25	73
21	Female	25	28	14
22	Male	46	28	82
23	Female	31	33	30
24	Female	49	33	73
25	Male	24	37	12
26	Male	49	38	94
27	Female	40	39	26
28	Male	23	39	65
29	Male	60	40	13
30	Female	21	40	73
31	Female	50	42	15
32	Female	24	42	65
33	Male	33	43	21
34	Male	59	43	79
35	Female	28	46	30
36	Female	35	46	87
37	Male	42	48	18
38	Female	26	48	75
39	Male	55	49	10
40	Female	32	49	85

ALGORITHM

1. Start the program.
2. Import libraries — `numpy`, `pandas`, `matplotlib`, and `KMeans` from `sklearn.cluster`.
3. Load the dataset using `pd.read\_csv('Mall\_Customers.csv')`.
4. Check missing values using `isnull().sum()`.
5. Handle missing data (if any).
6. Select features — *Annual Income* and *Spending Score* → store in variable `X`.
7. Create K-Means model with `n\_clusters = 5` and `init = 'k-means++'`.
8. Fit and predict clusters using `fit\_predict(X)`.
9. Plot data points for each cluster in different colors.
10. Mark centroids with yellow stars (`\*`).
11. Add title, labels, legend, and grid to the graph.
12. Stop the program.

PROGRAM

```
import numpy as np

import matplotlib.pyplot as plt

import pandas as pd

from sklearn.cluster import KMeans

dataset = pd.read_csv('Mall_Customers.csv')

print("Missing values in each column:\n", dataset.isnull().sum())

dataset = dataset.dropna(subset=['Annual Income (k$)', 'Spending Score (1-100)'])

X = dataset.iloc[:, [3, 4]].values

kmeans = KMeans(n_clusters=5, init='k-means++', random_state=42)

y_kmeans = kmeans.fit_predict(X)

plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s=100, c='red', label='Cluster 1')

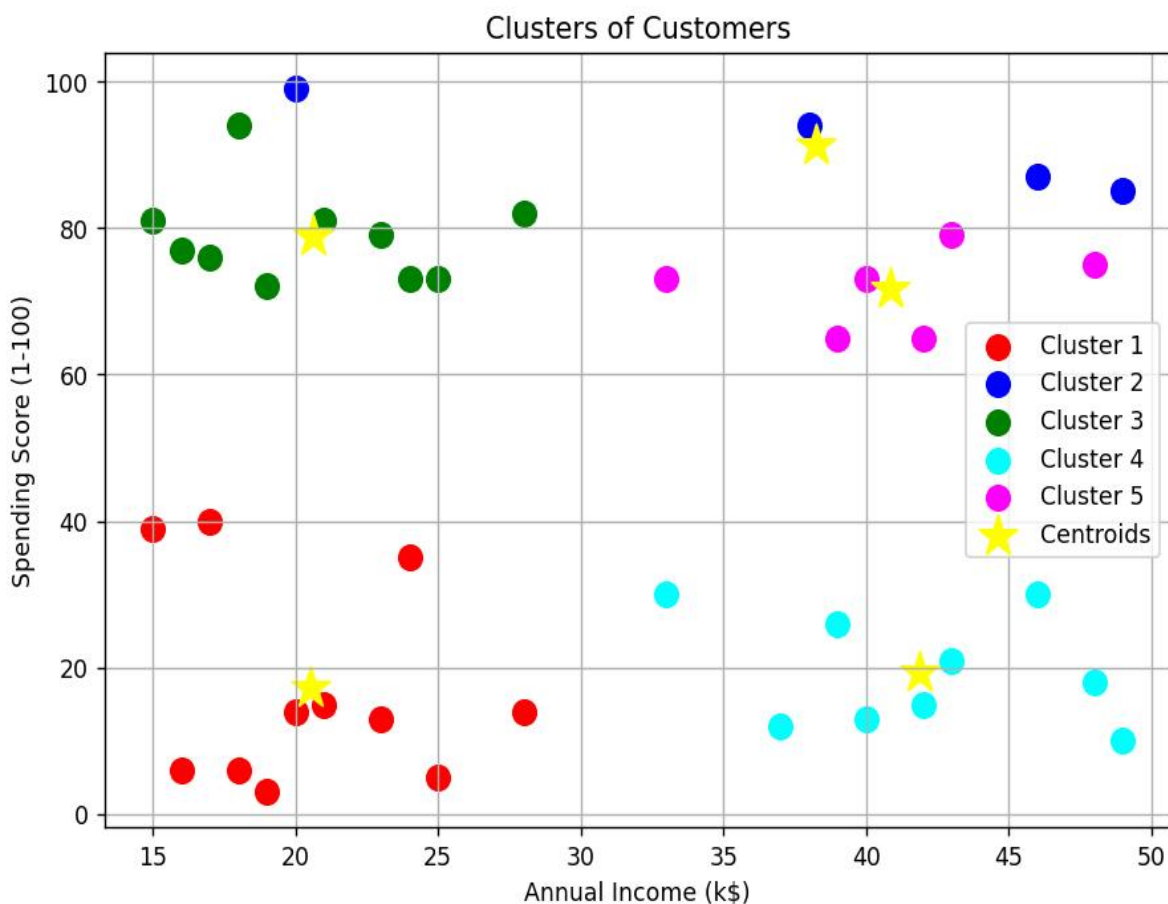
plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1], s=100, c='blue', label='Cluster 2')
```

```

plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1], s=100, c='green', label='Cluster 3')
plt.scatter(X[y_kmeans == 3, 0], X[y_kmeans == 3, 1], s=100, c='cyan', label='Cluster 4')
plt.scatter(X[y_kmeans == 4, 0], X[y_kmeans == 4, 1], s=100, c='magenta', label='Cluster 5')
plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1], s=300, c='yellow', marker='*',
label='Centroids')
plt.title('Clusters of Customers')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.grid(True)
plt.show()

```

OUTPUT



RESULT: Program executed successfully and output is verified.

PROGRAM 16

Convolutional Neural Network using Public Domain Dataset

AIM

To build and train a Convolutional Neural Network (CNN) using the Keras framework to classify images from the MNIST dataset.

ALGORITHM

1. Start
2. Import libraries like TensorFlow and Keras modules (datasets, layers, models).
3. Load the dataset: MNIST handwritten digits dataset.
4. Preprocess the data:
 - Reshape images to include channel dimension (28, 28, 1).
 - Normalize pixel values to the range [0,1].
5. Build the CNN model:
 - Add a Conv2D layer with ReLU activation.
 - Add a MaxPooling2D layer.
 - Add another Conv2D + MaxPooling2D layer.
 - Flatten the output.
 - Add a Dense layer with 64 neurons (ReLU).
 - Add output Dense layer with 10 neurons (softmax) for 10 classes.
6. Compile the model: Use Adam optimizer, sparse categorical cross-entropy loss, and accuracy metric.
7. Train the model on training data and validate using test data.
8. Evaluate the model on test data.
9. Print test accuracy.
10. Stop

PROGRAM

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models

(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()

x_train = x_train.reshape((x_train.shape[0], 28, 28, 1))
```

```

x_test = x_test.reshape((x_test.shape[0], 28, 28, 1))

x_train, x_test = x_train / 255.0, x_test / 255.0

model = models.Sequential([

layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),

layers.MaxPooling2D((2, 2)),

layers.Conv2D(64, (3, 3), activation='relu'),

layers.MaxPooling2D((2, 2)),

layers.Flatten(),

layers.Dense(64, activation='relu'),

layers.Dense(10, activation='softmax') # 10 classes (digits 0–9)])

model.compile(optimizer='adam',

loss='sparse_categorical_crossentropy',

metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))

test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)

print(f"Test Accuracy: {test_acc:.4f}")

```

OUTPUT

```

Epoch 1/5
1875/1875 ————— 34s 17ms/step - accuracy: 0.9563 - loss: 0.1431 - val_accuracy: 0.9831 - val_loss: 0.0507
Epoch 2/5
1875/1875 ————— 31s 17ms/step - accuracy: 0.9852 - loss: 0.0468 - val_accuracy: 0.9890 - val_loss: 0.0331
Epoch 3/5
1875/1875 ————— 42s 17ms/step - accuracy: 0.9898 - loss: 0.0329 - val_accuracy: 0.9899 - val_loss: 0.0286
Epoch 4/5
1875/1875 ————— 32s 17ms/step - accuracy: 0.9924 - loss: 0.0236 - val_accuracy: 0.9892 - val_loss: 0.0302
Epoch 5/5
1875/1875 ————— 31s 17ms/step - accuracy: 0.9944 - loss: 0.0173 - val_accuracy: 0.9904 - val_loss: 0.0311
313/313 - 2s - 8ms/step - accuracy: 0.9904 - loss: 0.0311
Test Accuracy: 0.9904

```

RESULT: Program executed successfully and output is verified.

PROGRAM - 17

Convolutional Neural Network using dataset

AIM

To implement a Convolutional Neural Network (CNN) for classifying tiny 4×4 grayscale images into two classes (Bright or Dark) using a custom dataset.

Pixel1	Pixel2	Pixel3	Pixel4	Pixel5	Pixel6	Pixel7	Pixel8	Pixel9
0.1	0.2	0.1	0.2	0.1	0.1	0.2	0.1	0.2
0.9	0.8	0.9	0.8	0.9	0.9	0.8	0.9	0.8
0.15	0.2	0.1	0.3	0.2	0.1	0.1	0.2	0.15
0.85	0.9	0.8	0.9	0.9	0.85	0.9	0.9	0.8
0.2	0.1	0.2	0.1	0.1	0.1	0.2	0.1	0.1
0.95	0.9	0.85	0.9	0.9	0.95	0.9	0.9	0.9

Pixel10	Pixel11	Pixel12	Pixel13	Pixel14	Pixel15	Pixel16	Label
0.1	0.1	0.2	0.1	0.1	0.2	0.1	Dark
0.9	0.9	0.8	0.9	0.8	0.9	0.9	Bright
0.1	0.2	0.1	0.2	0.15	0.1	0.1	Dark
0.9	0.9	0.8	0.9	0.85	0.9	0.9	Bright
0.2	0.1	0.1	0.2	0.1	0.1	0.2	Dark
0.9	0.85	0.9	0.9	0.9	0.9	0.95	Bright

ALGORITHM

1. Start
2. Import the required libraries like TensorFlow, Keras, Pandas, and Scikit-learn.
3. Load the dataset (simple_cnn_dataset.xlsx) into a pandas DataFrame.
4. Separate the input features (X) and output labels (y) from the dataset.
5. Encode the categorical labels into numeric form using LabelEncoder().
6. Reshape the input data into 4×4×1 format suitable for CNN input.
7. Split the dataset into training and testing sets using train_test_split().
8. Create a Sequential CNN model with the following layers:
 - Convolutional layer (Conv2D)
 - Flatten layer
 - Dense (fully connected) hidden layer
 - Output layer with sigmoid activation
9. Compile the model using Adam optimizer and binary crossentropy loss.

10. Train the model using the training data for a fixed number of epochs (e.g., 10).
11. Evaluate the model's accuracy using the test data.
12. Predict the class of a new sample and display whether it's "Bright" or "Dark."
13. Stop

PROGRAM

```
import tensorflow as tf

from tensorflow.keras import layers, models

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder

data = pd.read_excel('simple_cnn_dataset.xlsx')

X = data.drop('Label', axis=1).values

y = LabelEncoder().fit_transform(data['Label'])

X = X.reshape(-1, 4, 4, 1)

x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = models.Sequential([

    layers.Conv2D(4, (2, 2), activation='relu', input_shape=(4, 4, 1)),

    layers.Flatten(),

    layers.Dense(4, activation='relu'),

    layers.Dense(1, activation='sigmoid')

])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

model.fit(x_train, y_train, epochs=10, verbose=0)

loss, acc = model.evaluate(x_test, y_test, verbose=0)

print(f"\nTest Accuracy: {acc:.2f}")

pred = model.predict(x_test[:1])

print("Predicted Class:", "Bright" if pred[0][0] > 0.5 else "Dark")
```

OUTPUT

```
✓ Test Accuracy: 0.50  
1/1 ————— 0s 152ms/step  
Predicted Class: Dark
```

RESULT: Program executed successfully and output is verified.

PROGRAM -18

Web Crawler and Scrapping web pages

AIM

To implement a simple web crawler that visits a website and scrapes information from web pages.

ALGORITHM

1. Start
2. Import the required libraries requests and BeautifulSoup.
3. Set the target website URL.
4. Send a request to the URL using requests.get().
5. Get the HTML content of the page.
6. Parse the HTML using BeautifulSoup.
7. Find the main container that holds the required data.
8. Extract the job title, company name, and location from each job posting.
9. Display the extracted information.
10. Stop

PROGRAM

```
import requests

from bs4 import BeautifulSoup

URL = "https://realpython.github.io/fake-jobs/"

page = requests.get(URL)

print(page.text)

soup = BeautifulSoup(page.content, "html.parser")

results = soup.find(id="ResultsContainer")

job_elements = results.find_all("div", class_="card-content")

for job_element in job_elements:

    title_element = job_element.find("h2", class_="title")
```

```
company_element = job_element.find("h3", class_="company")  
location_element = job_element.find("p", class_="location")  
print(title_element.text.strip())  
print(company_element.text.strip())  
print(location_element.text.strip())  
print()
```

OUTPUT

```
Senior Python Developer  
Payne, Roberts and Davis  
Stewartbury, AA  
  
Energy engineer  
Vasquez-Davidson  
Christopherville, AA  
  
Legal executive  
Jackson, Chambers and Levy  
Port Ericaburgh, AA  
  
Fitness centre manager  
Savage-Bradley  
East Seanview, AP  
  
Product manager  
Ramirez Inc  
North Jamieview, AP  
  
Medical technical officer  
Rogers-Yates  
Davidville, AP  
  
Physiological scientist  
Kramer-Klein  
South Christopher, AE
```

RESULT: Program executed successfully and output is verified.

PROGRAM - 19

NLP Implementation using NLTK

AIM

To implement Natural Language Processing tasks such as Part-of-Speech tagging, N-gram generation, smoothing, and chunking using NLTK

ALGORITHM

1. Start.
2. Import NLTK libraries for tokenization, POS tagging, N-grams, stop words, stemming, smoothing, and chunking.
3. Download required NLTK resources (punkt, averaged_perceptron_tagger, stopwords).
4. Input the text to process.
5. Tokenize the text into words.
6. Remove stop words from the tokens.
7. Stem the filtered words to their root forms.
8. Perform Part-of-Speech (POS) tagging on the tokens.
9. Count occurrences of each POS tag.
10. Generate bigrams and trigrams from the tokens.
11. Calculate bigram frequencies.
12. Apply Lidstone smoothing to estimate probabilities of bigrams.
13. Define a grammar and chunk the tagged sentence to extract phrases (e.g., noun phrases).
14. Display all results (tokens, filtered words, stemmed words, POS tags, N-grams, frequencies, smoothed probabilities, chunked tree).
15. Stop.

PROGRAM

```
import nltk

from nltk import word_tokenize, pos_tag, bigrams, trigrams

from nltk.corpus import stopwords

from nltk.stem import PorterStemmer

from nltk.probability import FreqDist, LidstoneProbDist
```

```

from nltk import RegexpParser

from collections import Counter

nltk.data.path.append("C:/Desktop/nltk_data")

nltk.download('punkt', download_dir="C:/Desktop/nltk_data")

nltk.download('averaged_perceptron_tagger', download_dir="C:/Desktop/nltk_data")

nltk.download('stopwords', download_dir="C:/Desktop/nltk_data")

text = "Tomatoes are essential in cooking. They grow well in warm climates."

print("Original Text:\n", text)

tokens = word_tokenize(text)

print("\nTokens:\n", tokens)

stop_words = set(stopwords.words('english'))

filtered_words = [word for word in tokens if word.lower() not in stop_words and word.isalpha()]

print("\nAfter Removing Stop Words:\n", filtered_words)

ps = PorterStemmer()

stemmed_words = [ps.stem(word) for word in filtered_words]

print("\nAfter Stemming:\n", stemmed_words)

tags = pos_tag(tokens)

print("\nPart-of-Speech Tags:\n", tags)

tag_counts = Counter(tag for word, tag in tags)

print("\nPOS Tag Counts:\n", tag_counts)

bigrams_list = list(bigrams(tokens))

trigrams_list = list(trigrams(tokens))

print("\nBigrams:\n", bigrams_list)

print("\nTrigrams:\n", trigrams_list)

fdist = FreqDist(bigrams_list)

print("\nBigram Frequencies:")

for bigram, freq in fdist.items():

```

```

print(bigram, ":", freq)
lidstone_prob = LidstoneProbDist(fdist, 0.1, bins=len(fdist))
print("\nSmoothed Probability Example:")
print("P('Tomatoes', 'are') =", lidstone_prob.prob(('Tomatoes', 'are'))))
grammar = "NP: {<DT>?<JJ>*<NN>|<NNS>}"
chunk_parser = RegexpParser(grammar)
chunked = chunk_parser.parse(tags)
print("\nChunked Sentence Tree:")
print(chunked)

```

OUTPUT

```

Original Text:
Tomatoes are essential in cooking. They grow well in warm climates.

Tokens:
['Tomatoes', 'are', 'essential', 'in', 'cooking', '.', 'They', 'grow', 'well', 'in', 'warm', 'climates', '.']

After Removing Stop Words:
['Tomatoes', 'essential', 'cooking', 'grow', 'well', 'warm', 'climates']

After Stemming:
['tomato', 'essenti', 'cook', 'grow', 'well', 'warm', 'climat']

Part-of-Speech Tags:
[('Tomatoes', 'NNS'), ('are', 'VBP'), ('essential', 'JJ'), ('in', 'IN'), ('cooking', 'VBG'), ('.', '.'), ('They', 'PRP'), ('grow', 'VBP'), ('well', 'RB'), ('in', 'IN'), ('warm', 'JJ'), ('climates', 'NNS'), ('.', '.')]

POS Tag Counts:
Counter({'NNS': 2, 'VBP': 2, 'JJ': 2, 'IN': 2, '.': 2, 'VBG': 1, 'PRP': 1, 'RB': 1})

Bigrams:
[('Tomatoes', 'are'), ('are', 'essential'), ('essential', 'in'), ('in', 'cooking'), ('cooking', '.'), ('.', 'They'), ('They', 'grow'), ('grow', 'well'), ('well', 'in'), ('in', 'warm'), ('warm', 'climates'), ('climates', '.')]

Trigrams:
[('Tomatoes', 'are', 'essential'), ('are', 'essential', 'in'), ('essential', 'in', 'cooking'), ('in', 'cooking', '.'), ('cooking', '.', 'They'), ('.', 'The', 'y'), ('grow', 'They', 'grow'), ('well', 'grow', 'well'), ('grow', 'well', 'in'), ('well', 'in', 'warm'), ('in', 'warm', 'climates'), ('warm', 'climates', '.')]

```

```

Bigram Frequencies:
('Tomatoes', 'are') : 1
('are', 'essential') : 1
('essential', 'in') : 1
('in', 'cooking') : 1
('cooking', '.') : 1
('.', 'They') : 1
('They', 'grow') : 1
('grow', 'well') : 1
('well', 'in') : 1
('in', 'warm') : 1
('warm', 'climates') : 1
('climates', '.') : 1

Smoothed Probability Example:
P('Tomatoes', 'are') = 0.08333333333333334

Chunked Sentence Tree:
(S
  (NP Tomatoes/NNS)
  are/VBP
  essential/JJ
  in/IN
  cooking/VBG
  ./
  They/PRP
  grow/VBP
  well/RB
  in/IN
  warm/JJ
  (NP climates/NNS)
  ./)

```

RESULT: Program executed successfully and output is verified.

